Online Retail II

Data Warehouse

ETL Process, OLAP Cube & Power BI Report

Database Source: UCI Online Retail II Dataset
Business Area: Sales Performance Analysis

Group Members: Aly Hassan, Mohamed Ehab,
Alaa Shaaban, Kenzy Zedan, Youssef Osama.

Course: DE324: Big Data Engineering

Semester: Spring 2025

Table of Contents

1. Introduction 3
   1.1 Business Goals 3
   1.2 Data Source 3
2. Data Warehouse Schema 4
   2.1 Tables Overview 4
   2.2 SQL — Create Tables 5
   2.3 Populate DIM_DATE 5
   3.1 Source Tables Used 7
   3.2 Step 1 — Cleanup (Execute SQL Task) 7
   3.3 Step 2 — Load Staging 7
   3.4 Step 4 — Load DIM_COUNTRY 8
   3.5 Step 5 — Load DIM_PRODUCT 8
   3.6 Step 6 — Load DIM_CUSTOMER 9
   3.7 Step 7 — Load FACT_SALES 10
4. Data Quality & Transformations 12
   4.1 Handling Nulls 12
   4.2 Handling Duplicates 12
   4.3 Cancelled Invoices & Returns 12
   4.4 Surrogate Key Generation 12
   4.5 Slowly Changing Dimensions (SCD) 12
5. Data Verification & Validation 14
   5.1 Row Count Verification 14
   5.2 Referential Integrity Validation 15
   5.3 Star Schema Filled with Data 15
6. OLAP Cube (SSAS) 19
   6.1 Data Source & Data Source View 19
   6.2 Dimensions 19
   6.3 Measures (FACT_SALES) 19
7. MDX Queries 20
   7.1 Top 5 Products — Revenue + Quantity + Price 20
   7.2 Sales Over Time (Yearly) 20
   7.3 Revenue Share % per Country Group 21
   7.4 Average Order Value Over the Year 21
   7.5 How Do Sales Vary Over Time Monthly (last 6 months 2011) 22
8. Power BI Report & Dashboard 23
   8.1 Connection 23
   8.2 Visuals Created 23
9. Summary 26

1. Introduction

This document describes the complete process of building a Data Warehouse for the Online Retail II dataset. The goal is to analyze sales performance and support business decision-making. The pipeline covers four main stages: database setup in SQL Server Management Studio (SSMS), data loading and transformation using SQL Server Integration Services (SSIS), cube creation using SQL Server Analysis Services (SSAS), and dashboard development in Power BI.

1.1 Business Goals

The data warehouse was built to answer the following business questions:

- What are the top-selling products?
- How do sales vary over time (monthly, yearly)?
- Which customers or countries generate the most revenue?
- What is the average order value?

1.2 Data Source

The source data is the Online Retail II dataset from the UCI Machine Learning Repository. It is an Excel file (xlsx) containing two sheets covering transactions from 2009 to 2011. The dataset includes invoice records, product stock codes, descriptions, quantities, prices, customer IDs, and countries.

Source Link: https://archive.ics.uci.edu/dataset/502/online+retail+ii

	A	B	C	D	E	F	G	H
1	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
2	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	12/1/10 8:26	2.55	17850	United Kingdom
3	536365	71053	WHITE METAL LANTERN	6	12/1/10 8:26	3.39	17850	United Kingdom
4	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	12/1/10 8:26	2.75	17850	United Kingdom
5	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	12/1/10 8:26	3.39	17850	United Kingdom
6	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	12/1/10 8:26	3.39	17850	United Kingdom
7	536365	22752	SET 7 BABUSHKA NESTING BOXES	2	12/1/10 8:26	7.65	17850	United Kingdom
8	536365	21730	GLASS STAR FROSTED T-LIGHT HOLDER	6	12/1/10 8:26	4.25	17850	United Kingdom

Figure 1- Sample Data from The .xlsx file

2. Data Warehouse Schema

A Star Schema was designed with one central fact table and four surrounding dimension tables. The schema was created in SQL Server (SSMS) on the OnlineRetailDW database. The Star Schema was altered, referencing the DIM_COUNTRY table directly to the fact table instead of linking it to the customer dimension, which improves the queries' performance.

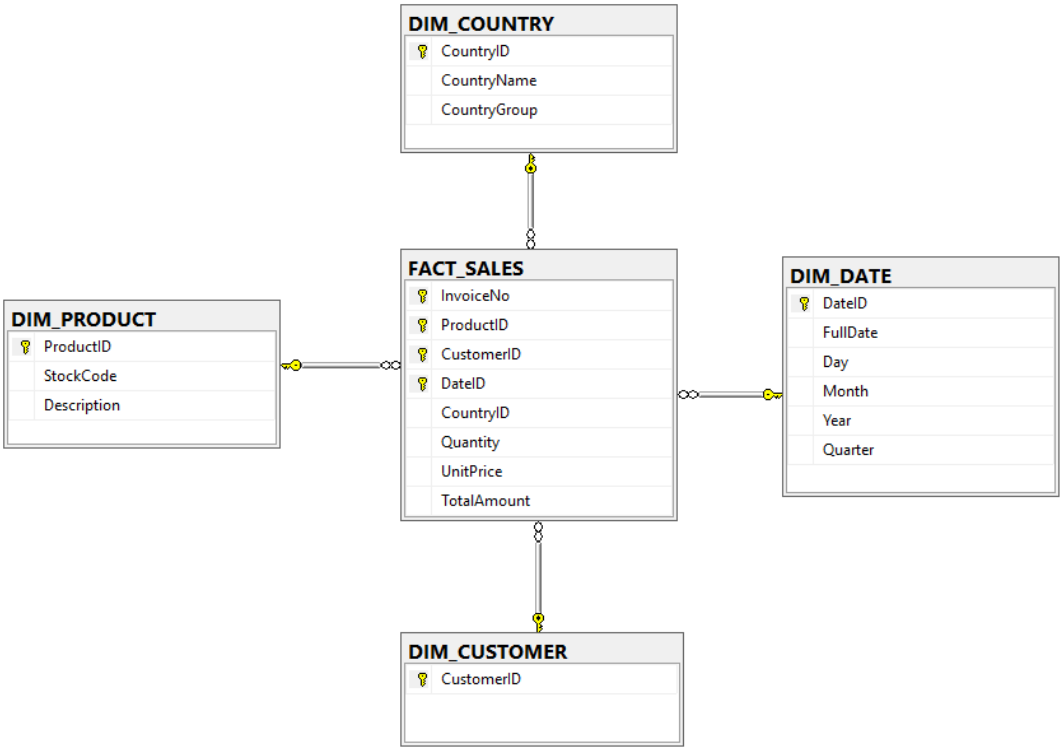

Figure 2- Modified Star Schema

2.1 Tables Overview

Table	Type	Description
STG_OnlineRetail	Staging	Raw data from XLSX before transformation
DIM_COUNTRY	Dimension	Country names and regional groupings
DIM_PRODUCT	Dimension	Product stock codes and descriptions
DIM_CUSTOMER	Dimension	Unique customer IDs
DIM_DATE	Dimension	Date breakdown: day, month, year, quarter
FACT_SALES	Fact	Sales transactions with measures and FK references

2.2 SQL — Create Tables

The following SQL was executed in SSMS to create the data warehouse tables:

```
-- DIM_COUNTRY
CREATE TABLE DIM_COUNTRY (
    CountryID    INT           IDENTITY(1,1) PRIMARY KEY,
    CountryName  NVARCHAR(255) NOT NULL,
    CountryGroup NVARCHAR(100) NOT NULL
);

-- DIM_CUSTOMER
CREATE TABLE DIM_CUSTOMER (
    CustomerID INT NOT NULL PRIMARY KEY
);

-- DIM_PRODUCT
CREATE TABLE DIM_PRODUCT (
    ProductID   INT           IDENTITY(1,1) PRIMARY KEY,
    StockCode   NVARCHAR(255) NOT NULL UNIQUE,
    Description NVARCHAR(500) NOT NULL
);

-- DIM_DATE
CREATE TABLE DIM_DATE (
    DateID   INT      PRIMARY KEY,
    FullDate DATE     NOT NULL,
    Day      TINYINT  NOT NULL,
    Month    TINYINT  NOT NULL,
    Year     SMALLINT NOT NULL,
    Quarter  TINYINT  NOT NULL
);

-- FACT_SALES
CREATE TABLE FACT_SALES (
    InvoiceNo   NVARCHAR(20) NOT NULL,
    ProductID   INT          NOT NULL FOREIGN KEY REFERENCES
DIM_PRODUCT(ProductID),
    CustomerID  INT          NOT NULL FOREIGN KEY REFERENCES
DIM_CUSTOMER(CustomerID),
    DateID      INT          NOT NULL FOREIGN KEY REFERENCES DIM_DATE(DateID),
    CountryID   INT          NOT NULL FOREIGN KEY REFERENCES
DIM_COUNTRY(CountryID),
    Quantity    INT          NOT NULL,
    UnitPrice   FLOAT        NOT NULL,
    TotalAmount AS (CAST(Quantity AS FLOAT) * UnitPrice),
    PRIMARY KEY (InvoiceNo, ProductID, CustomerID, DateID)
);
```

2.3 Populate DIM_DATE

DIM_DATE is populated using a recursive CTE that generates one row per day from January 2009 to December 2011, covering the full dataset range:

```
WITH DateCTE AS (
    SELECT CAST('2009-01-01' AS DATE) AS d
    UNION ALL
    SELECT DATEADD(DAY, 1, d) FROM DateCTE WHERE d < '2012-12-31'
)
INSERT INTO DIM_DATE (DateID, FullDate, Day, Month, Year, Quarter)
```

```sql
SELECT
    CONVERT(INT, FORMAT(d, 'yyyyMMdd')),
    d, DAY(d), MONTH(d), YEAR(d), DATEPART(QUARTER, d)
FROM DateCTE
OPTION (MAXRECURSION 2000);
```

3. ETL Process (SSIS)

The ETL (Extract, Transform, Load) process was built using SQL Server Integration Services (SSIS) in Visual Studio 2022. The SSIS package contains a Control Flow with multiple Data Flow Tasks, each responsible for loading a specific table. The overall flow is:

- Truncate and Delete all tables (cleanup if restart needed) - Disabled later
- Load Both Sheets to Staging (from .xlsx using Union All)
- Load DIM_COUNTRY from Staging
- Load DIM_PRODUCT from Staging
- Load DIM_CUSTOMER from Staging
- Load FACT_SALES from Staging (with Lookups for ProductID and CountryID)

3.1 Source Tables Used

The only source is the staging table STG_OnlineRetail, which is populated from the two Excel sheets in the Online Retail II XLSX file. The two sheets (Year 2009-2010 and Year 2010-2011) are loaded in a single Data Flow Task using a Union All transformation.

3.2 Step 1 — Cleanup (Execute SQL Task)

Before every run, an Execute SQL Task at the start of the Control Flow clears all tables to prevent duplicate data:

```
TRUNCATE TABLE STG_OnlineRetail;
DELETE FROM FACT_SALES;
DELETE FROM DIM_CUSTOMER;
DELETE FROM DIM_PRODUCT;
DELETE FROM DIM_COUNTRY;
```

3.3 Step 2 — Load Staging

Both Excel sheets are loaded into the staging table using a single Data Flow Task. Two Excel Sources (one per sheet) feed into a Union All transformation, which sends all rows to the OLE DB Destination (STG_OnlineRetail). The connection forces all columns to be read as text, preventing the Excel rows from being misread, mixed alphanumeric values like stock codes (e.g., 48173C) as NULL.

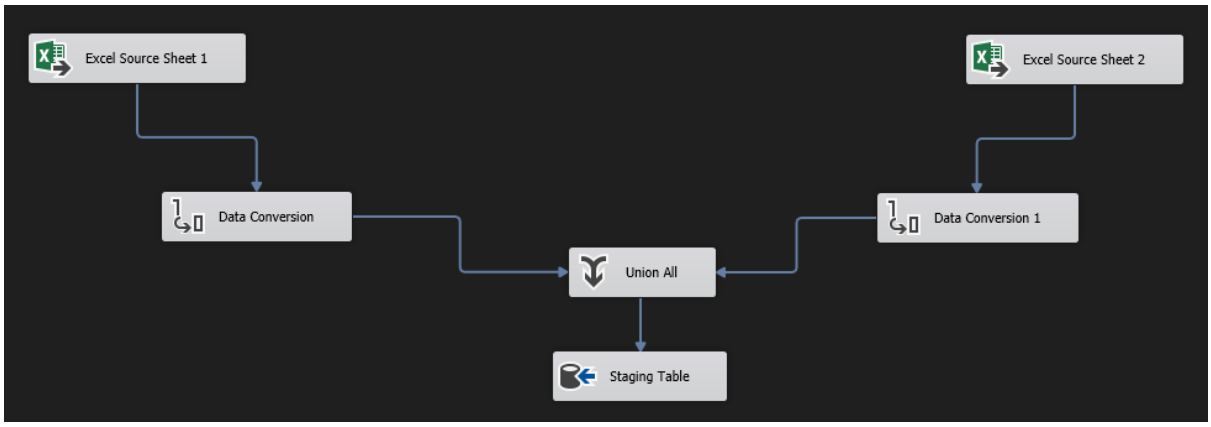

Figure 3- Data Extraction Data Flow Task

3.4 Step 4 — Load DIM_COUNTRY

DIM_COUNTRY is loaded from staging using a SQL query that selects distinct country names. A Derived Column transformation adds the CountryGroup column, which categorizes each country into a regional group (e.g., British Isles, Europe, Americas, Asia Pacific, Middle East, Africa, Other). This is an enriched/derived column not present in the source data, added to support regional sales analysis.

```
SELECT DISTINCT
    LTRIM(RTRIM(s.Country)) AS CountryName
FROM STG_OnlineRetail s
WHERE s.Country IS NOT NULL
  AND LTRIM(RTRIM(s.Country)) <> ''
  AND NOT EXISTS (
      SELECT 1
      FROM DIM_COUNTRY d
      WHERE d.CountryName = LTRIM(RTRIM(s.Country))
  )
```

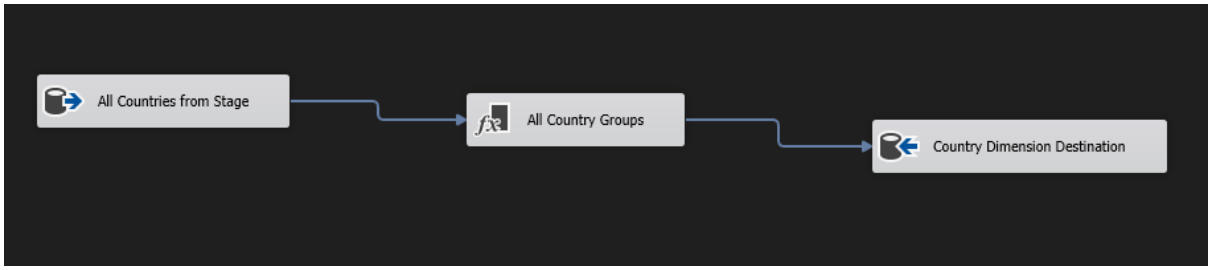

Figure 4- Staging To DIM_COUNTRY Data Flow Task

3.5 Step 5 — Load DIM_PRODUCT

DIM_PRODUCT is loaded using GROUP BY with MAX(Description) to ensure only one description per StockCode is inserted, avoiding duplicate key violations. Rows with NULL or blank StockCode and Description are excluded.

```
SELECT
```

```
    LTRIM(RTRIM(s.StockCode)) AS StockCode,
    MAX(LTRIM(RTRIM(s.Description))) AS Description
FROM STG_OnlineRetail s
WHERE s.StockCode IS NOT NULL
  AND LTRIM(RTRIM(s.StockCode)) <> ''
  AND s.Description IS NOT NULL
  AND LTRIM(RTRIM(s.Description)) <> ''
  AND NOT EXISTS (
      SELECT 1
      FROM DIM_PRODUCT d
      WHERE d.StockCode = LTRIM(RTRIM(s.StockCode))
  )
GROUP BY LTRIM(RTRIM(s.StockCode))
```

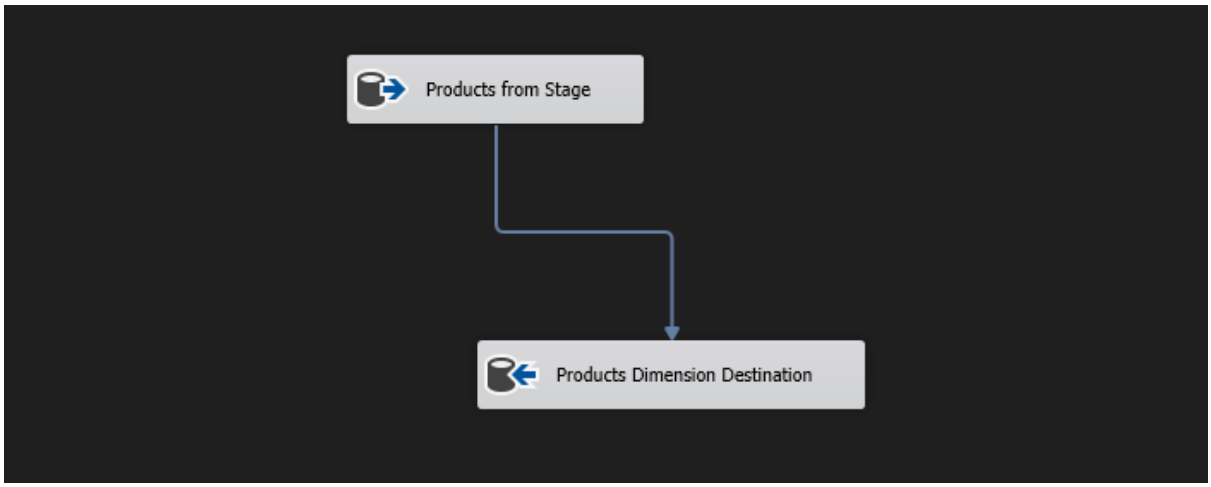

Figure 5- Staging To DIM_PRODUCT Data Flow Task

3.6 Step 6 — Load DIM_CUSTOMER

DIM_CUSTOMER contains only CustomerID. The query groups by CustomerID to deduplicate customers who appear across both sheets. Rows with NULL or non-numeric CustomerID are excluded using TRY_CAST.

```
SELECT
    TRY_CAST(s.CustomerID AS INT) AS CustomerID,
    MAX(LTRIM(RTRIM(s.Country))) AS CountryName
FROM STG_OnlineRetail s
WHERE s.CustomerID IS NOT NULL
  AND LTRIM(RTRIM(s.CustomerID)) <> ''
  AND TRY_CAST(s.CustomerID AS INT) IS NOT NULL
  AND NOT EXISTS (
      SELECT 1
      FROM DIM_CUSTOMER d
      WHERE d.CustomerID = TRY_CAST(s.CustomerID AS INT)
  )
GROUP BY TRY_CAST(s.CustomerID AS INT)
```

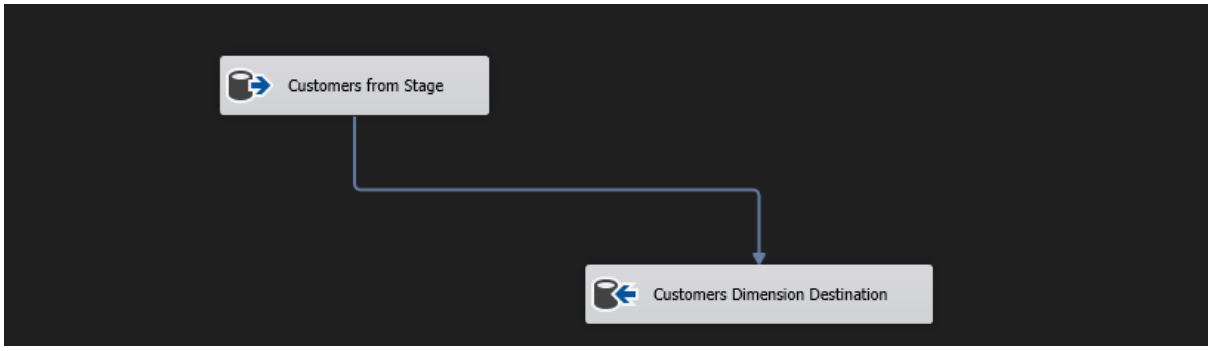

Figure 6- Staging To DIM_CUSTOMERS Data Flow Task

3.7 Step 7 — Load FACT_SALES

FACT_SALES is the most complex load. The source query reads from staging with all cleaning filters applied. Two Lookup transformations resolve ProductID from DIM_PRODUCT (matching on StockCode) and CountryID from DIM_COUNTRY (matching on CountryName). TotalAmount is a computed column in SQL Server and is left as ignored in SSIS.

```
SELECT
    LTRIM(RTRIM(s.Invoice)) AS InvoiceNo,
    LTRIM(RTRIM(s.StockCode)) AS StockCode,
    TRY_CAST(s.CustomerID AS INT) AS CustomerID,
    CONVERT(INT, FORMAT(TRY_CAST(s.InvoiceDate AS DATE), 'yyyyMMdd')) AS
DateID,
    LTRIM(RTRIM(s.Country)) AS CountryName,
    SUM(s.Quantity) AS Quantity,
    MAX(s.Price) AS UnitPrice
FROM STG_OnlineRetail s
WHERE s.Invoice IS NOT NULL
  AND s.CustomerID IS NOT NULL
  AND TRY_CAST(s.CustomerID AS INT) IS NOT NULL
  AND s.Quantity > 0
  AND s.Price > 0
  AND TRY_CAST(s.InvoiceDate AS DATETIME) IS NOT NULL
  AND LEFT(LTRIM(RTRIM(s.Invoice)), 1) <> 'C'
  AND NOT EXISTS (
      SELECT 1
      FROM FACT_SALES f
      WHERE f.InvoiceNo = LTRIM(RTRIM(s.Invoice))
        AND f.CustomerID = TRY_CAST(s.CustomerID AS INT)
        AND f.DateID = CONVERT(INT, FORMAT(TRY_CAST(s.InvoiceDate AS DATE),
'yyyyMMdd'))
  )
GROUP BY
    LTRIM(RTRIM(s.Invoice)),
    LTRIM(RTRIM(s.StockCode)),
    TRY_CAST(s.CustomerID AS INT),
    CONVERT(INT, FORMAT(TRY_CAST(s.InvoiceDate AS DATE), 'yyyyMMdd')),
    LTRIM(RTRIM(s.Country))
```

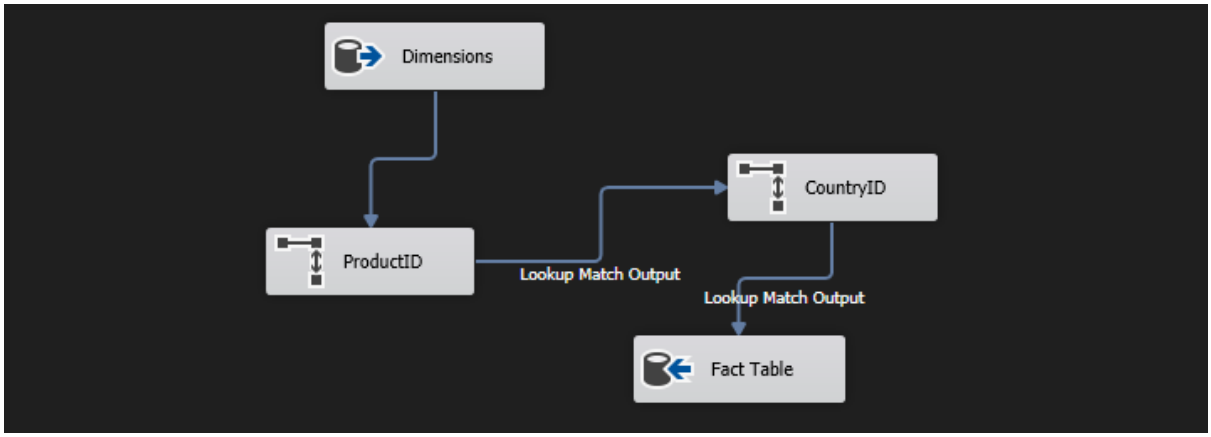

Figure 7- DIM_COUNTRY to SALES_FACT Data Flow Task

A NOT EXISTS subquery was added to the load query to prevent duplicate key violations when the pipeline is rerun without truncating first. This makes the load incremental — only rows that do not already exist in FACT_SALES based on InvoiceNo, CustomerID, and DateID are inserted, allowing new data to be added safely without affecting existing records.

4. Data Quality & Transformations

4.1 Handling Nulls

Null values were handled at the FACT_SALES loading stage using WHERE clause filters:

- Rows with NULL Invoice were excluded
- Rows with NULL or non-numeric CustomerID were excluded using TRY_CAST
- Rows with NULL InvoiceDate were excluded using TRY_CAST
- Rows with NULL or blank StockCode/Description were excluded from DIM_PRODUCT
- 80,112 rows had NULL StockCode due to Excel driver type-guessing and were documented as excluded

4.2 Handling Duplicates

Duplicates were handled differently for each table:

- DIM_PRODUCT: GROUP BY StockCode with MAX(Description) ensures one product per StockCode
- DIM_CUSTOMER: GROUP BY CustomerID ensures one row per customer
- DIM_COUNTRY: DISTINCT on CountryName ensures no duplicate countries
- FACT_SALES: GROUP BY on all key columns with SUM(Quantity) and MAX(Price) prevents duplicate composite primary key violations

4.3 Cancelled Invoices & Returns

Cancelled invoices in the source data are identified by an Invoice number starting with the letter 'C'. These were excluded using the filter: LEFT(LTRIM(Invoice), 1) <> 'C'. Additionally, rows with Quantity <= 0 (returns) and Price <= 0 were also excluded to ensure only valid forward sales are recorded in the fact table.

4.4 Surrogate Key Generation

Table	Key Type	Method
DIM_COUNTRY	Surrogate Key	IDENTITY(1,1) — auto-generated by SQL Server
DIM_PRODUCT	Surrogate Key	IDENTITY(1,1) — auto-generated by SQL Server
DIM_CUSTOMER	Natural Key	CustomerID from source — already a stable unique integer
DIM_DATE	Smart Key	YYYYMMDD integer format — e.g. 20101201

4.5 Slowly Changing Dimensions (SCD)

During the ETL process, it was identified that some CustomerIDs appear with different Country values across the two Excel sheets. This indicates a Slowly Changing Dimension scenario. SCD Type 1 (overwrite) was chosen as the handling strategy. The most recent country value is retained and the old one is overwritten. This decision was made because tracking the historical country of a customer is not relevant to the business goals of this project, which focus on sales performance and revenue analysis rather than customer location history.

5. Data Verification & Validation

After loading, the following queries were run in SSMS to verify row counts and referential integrity across all tables.

5.1 Row Count Verification

```
SELECT 'STG_OnlineRetail' AS TableName, COUNT(*) AS RowCnt FROM
STG_OnlineRetail
UNION ALL SELECT 'DIM_COUNTRY',  COUNT(*) FROM DIM_COUNTRY
UNION ALL SELECT 'DIM_PRODUCT',  COUNT(*) FROM DIM_PRODUCT
UNION ALL SELECT 'DIM_CUSTOMER', COUNT(*) FROM DIM_CUSTOMER
UNION ALL SELECT 'DIM_DATE',     COUNT(*) FROM DIM_DATE
UNION ALL SELECT 'FACT_SALES',   COUNT(*) FROM FACT_SALES;
```

Table	Row Count
STG_OnlineRetail	1,067,371
DIM_COUNTRY	43
DIM_PRODUCT	4,376
DIM_CUSTOMER	5,942
DIM_DATE	1,461
FACT_SALES	714,382

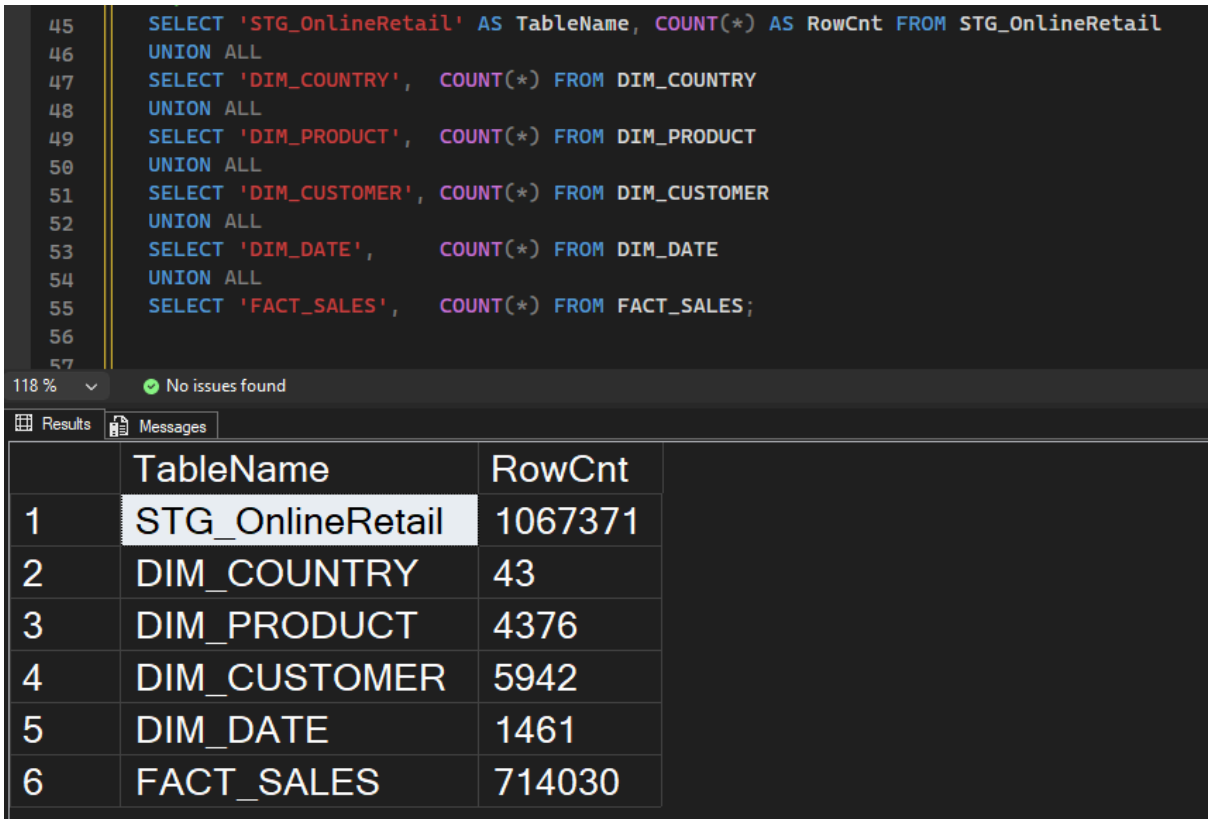

Figure 8- Row Count Verification

5.2 Referential Integrity Validation

The following queries verify that no orphaned records exist in the fact table — all FK values must exist in their respective dimension tables. All queries returned 0, confirming data integrity.

```
-- Orphaned CustomerIDs
SELECT COUNT(*) AS OrphanedCustomers FROM FACT_SALES f
WHERE NOT EXISTS (SELECT 1 FROM DIM_CUSTOMER c WHERE c.CustomerID =
f.CustomerID);

-- Orphaned ProductIDs
SELECT COUNT(*) AS OrphanedProducts FROM FACT_SALES f
WHERE NOT EXISTS (SELECT 1 FROM DIM_PRODUCT p WHERE p.ProductID = f.ProductID);

-- Orphaned DateIDs
SELECT COUNT(*) AS OrphanedDates FROM FACT_SALES f
WHERE NOT EXISTS (SELECT 1 FROM DIM_DATE d WHERE d.DateID = f.DateID);

-- Orphaned CountryIDs
SELECT COUNT(*) AS OrphanedCountries FROM FACT_SALES f
WHERE NOT EXISTS (SELECT 1 FROM DIM_COUNTRY c WHERE c.CountryID = f.CountryID);
```

	OrphanedCustomers
1	0

	OrphanedProducts
1	0

	OrphanedDates
1	0

	OrphanedCountries
1	0

Figure 9- Referential Integrity Validation

5.3 Star Schema Filled with Data

The following screenshots show sample data from each table after the ETL process is completed.

```
79  SELECT TOP 10 * FROM DIM_COUNTRY
80
81
82
```

230 % No issues found

Results Messages

	Country...	CountryName	CountryGroup
1	904	Finland	Europe
2	905	Bahrain	Middle East
3	906	EIRE	British Isles
4	907	USA	Americas
5	908	Italy	Europe
6	909	Brazil	Americas
7	910	Bermuda	Other
8	911	Netherlands	Europe
9	912	Hong Kong	Asia Pacific
10	913	Germany	Europe

Figure 10- DIM_COUNTRY Sample Data

```
83  SELECT TOP 10 * FROM DIM_PRODUCT
84
85
86
```

230 % No issues found

Results Messages

	ProductID	StockCode	Description
1	80157	22423	wonky bottom/broken
2	80158	22515	CHILDS GARDEN SPADE PINK
3	80159	22758	LARGE PURPLE BABUSHKA NOTEBOOK
4	80160	21000	ROSE DU SUD COSMETICS BAG
5	80161	82093	HEART SHAPE BUTTON FRAME
6	80162	21659	GLASS CAKE STAND
7	80163	85165	damages
8	80164	22861	EASTER TIN CHICKS IN GARDEN
9	80165	22633	HAND WARMER UNION JACK
10	80166	82597	BOMBS AWAY METAL SIGN

Figure 11- DIM_PRODUCT Sample Data

```
89  SELECT TOP 10 * FROM DIM_CUSTOMER
90
91
92
93
```

230 % No issues found

Results Messages

	CustomerID
1	12346
2	12347
3	12348
4	12349
5	12350
6	12351
7	12352
8	12353
9	12354
10	12355

Figure 12- DIM_CUSTOMER Sample Data

```
94  SELECT TOP 10 * FROM DIM_DATE
95
96
97
```

230 % No issues found

Results Messages

	DateID	FullDate	Day	Month	Year	Quarter
1	20090101	2009-01-01	1	1	2009	1
2	20090102	2009-01-02	2	1	2009	1
3	20090103	2009-01-03	3	1	2009	1
4	20090104	2009-01-04	4	1	2009	1
5	20090105	2009-01-05	5	1	2009	1
6	20090106	2009-01-06	6	1	2009	1
7	20090107	2009-01-07	7	1	2009	1
8	20090108	2009-01-08	8	1	2009	1
9	20090109	2009-01-09	9	1	2009	1
10	20090110	2009-01-10	10	1	2009	1

Figure 13- DIM_DATE Sample Data

```
SELECT TOP 10 * FROM FACT_SALES
```

230 % No issues found

Results Messages

	InvoiceNo	ProductID	CustomerID	DateID	CountryID	Quantity	UnitPrice	TotalAmount
1	489434	82324	13085	20091201	927	24	1.65	39.6
2	489434	82769	13085	20091201	927	12	6.95	83.4
3	489434	83443	13085	20091201	927	48	2.1	100.8
4	489434	84102	13085	20091201	927	24	1.25	30
5	489434	84115	13085	20091201	927	24	1.25	30
6	489434	84150	13085	20091201	927	10	5.95	59.5
7	489435	80800	13085	20091201	927	12	2.55	30.6
8	489435	83251	13085	20091201	927	12	2.55	30.6
9	489435	83338	13085	20091201	927	24	1.65	39.6
10	489435	83992	13085	20091201	927	12	3.75	45

Figure 14- FACT_SALES Sample Data

6. OLAP Cube (SSAS)

An OLAP cube was built using SQL Server Analysis Services (SSAS) 2022 in Multidimensional mode. The cube connects to the OnlineRetailDW database on the SQLEXPRESS instance and enables multi-dimensional analysis of sales data.

6.1 Data Source & Data Source View

A Data Source was created pointing to the OnlineRetailDW database on localhost\SQLEXPRESS using Windows Authentication with TrustServerCertificate=True. A Data Source View was created including all five warehouse tables with their relationships.

6.2 Dimensions

Dimension	Key Attribute	Hierarchy
DIM_COUNTRY	Country ID	Country Group -> Country Name
DIM_PRODUCT	Product ID	No hierarchy
DIM_CUSTOMER	Customer ID	No hierarchy (single attribute)
DIM_DATE	Date ID	Year -> Month -> Day -> Full Date

6.3 Measures (FACT_SALES)

- Quantity — total units sold
- Unit Price — price per unit
- Total Amount — computed as Quantity x UnitPrice
- Fact Sales Count — number of transactions

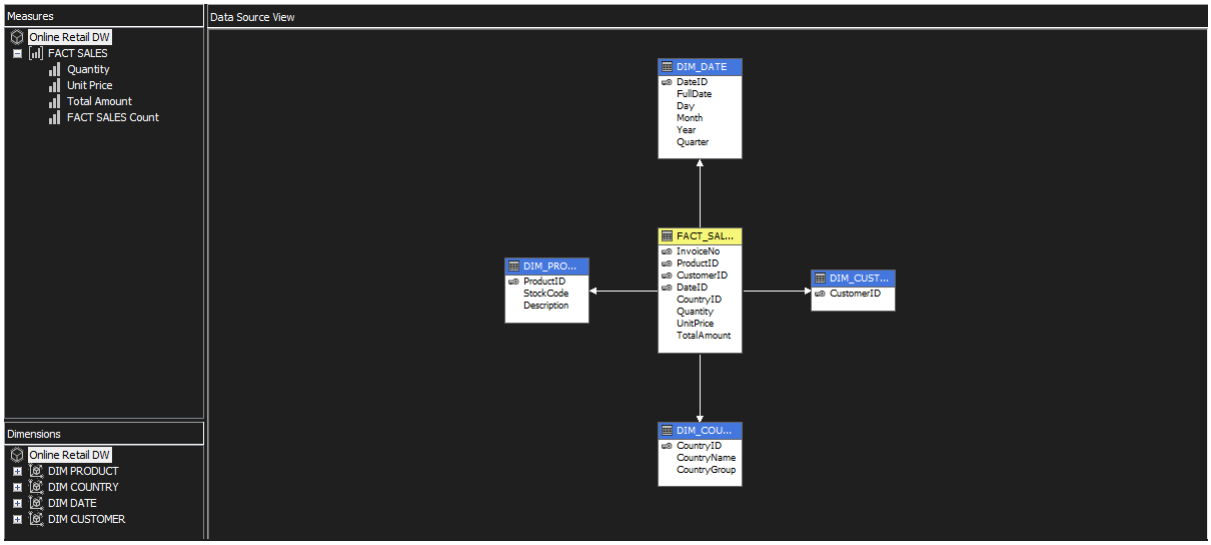

Figure 15- Online Retail DW Cube

7. MDX Queries

The following MDX queries were written and executed against the OLAP cube to answer the five business goals. Each query was run in SSMS connected to the Analysis Services instance on localhost.

7.1 Top 5 Products — Revenue + Quantity + Price

```
SELECT
  NON EMPTY
    {
      [Measures].[Total Amount],
      [Measures].[Quantity],
      [Measures].[Unit Price]
    } ON COLUMNS,
  NON EMPTY
    TOPCOUNT(
      [DIM PRODUCT].[Stock Code].[Stock Code].MEMBERS,
      5,
      [Measures].[Total Amount]
    ) ON ROWS
FROM [Online Retail DW]
```

Messages | Results

	Total Amount	Quantity	Unit Price
22423	286515.1	24899	41331.9
23843	168469.6	80995	2.08
84879	127074.17	79913	4444.59999999999
47566	103925.23	23607	9914.42000000003
85123A	100603.5	36782	5719.32000000002

Figure 16- MDX Query 1 Output - Top 5 Products — Revenue + Quantity + Price

7.2 Sales Over Time (Yearly)

```
SELECT
  NON EMPTY [DIM DATE].[Year].[Year].MEMBERS ON ROWS,
  {[Measures].[Total Amount]} ON COLUMNS
FROM [Online Retail DW]
```

Messages | Results

	Total Amount
2009	505448.600000009
2010	7408473.02999936
2011	8330927.92299958

Figure 17- MDX Query 2 Output - Sales Over Time (Yearly)

7.3 Revenue Share % per Country Group

```
WITH
  MEMBER [Measures].[Country Group Revenue %] AS
    IIF(
      ( [DIM COUNTRY].[Country Group].DEFAULTMEMBER,
        [Measures].[Total Amount] ) = 0,
      NULL,
      [Measures].[Total Amount]
      / ( [DIM COUNTRY].[Country Group].DEFAULTMEMBER,
          [Measures].[Total Amount] )
    ),
    FORMAT_STRING = "Percent"
SELECT
  NON EMPTY
    {
      [Measures].[Total Amount],
      [Measures].[Country Group Revenue %]
    } ON COLUMNS,
  NON EMPTY
    [DIM COUNTRY].[Country Group].[Country Group].MEMBERS ON ROWS
FROM [Online Retail DW]
```

Messages | Results

	Total Amount	Country Group Revenue %
Africa	1858.86	0.01%
Americas	14011.66	0.09%
Asia Pacific	248474.04	1.53%
British Isles	14080015.572999	86.67%
Europe	1870489.66999999	11.51%
Middle East	21144.66	0.13%
Other	8855.09	0.05%

Figure 18- MDX Query 3 Output – Revenue Share % per Country Group

7.4 Average Order Value Over the Year

```
SELECT
  {[Measures].[Total Amount], [Measures].[Fact Sales Count]} ON COLUMNS,
  NON EMPTY [DIM DATE].[Year].[Year].MEMBERS ON ROWS
FROM [Online Retail DW]
```

	Total Amount	FACT SALES Count
2009	505448.600000009	23677
2010	7408473.02999936	328867
2011	8330927.92299958	361713

Figure 19- MDX Query 4 Output – Average Order Value Over the Year

7.5 How Do Sales Vary Over Time Monthly (last 6 months 2011)

```
WITH
  MEMBER [Measures].[Jul 2011] AS
    ([DIM DATE].[Hierarchy].[Month].&[2011]&[7], [Measures].[Total Amount])
  MEMBER [Measures].[Aug 2011] AS
    ([DIM DATE].[Hierarchy].[Month].&[2011]&[8], [Measures].[Total Amount])
  MEMBER [Measures].[Sep 2011] AS
    ([DIM DATE].[Hierarchy].[Month].&[2011]&[9], [Measures].[Total Amount])
  MEMBER [Measures].[Oct 2011] AS
    ([DIM DATE].[Hierarchy].[Month].&[2011]&[10], [Measures].[Total Amount])
  MEMBER [Measures].[Nov 2011] AS
    ([DIM DATE].[Hierarchy].[Month].&[2011]&[11], [Measures].[Total Amount])
  MEMBER [Measures].[Dec 2011] AS
    ([DIM DATE].[Hierarchy].[Month].&[2011]&[12], [Measures].[Total Amount])
SELECT
  NON EMPTY {
    [Measures].[Jul 2011],
    [Measures].[Aug 2011],
    [Measures].[Sep 2011],
    [Measures].[Oct 2011],
    [Measures].[Nov 2011],
    [Measures].[Dec 2011]
  } ON COLUMNS
FROM [Online Retail DW]
```

Messages | Results

Jul 2011	Aug 2011	Sep 2011	Oct 2011	Nov 2011	Dec 2011
599099.041000014	643768.790000007	951514.050999997	1040196.69	1160792.97000003	517694.950000002

Figure 20- MDX Query 5 Output – How Do Sales Vary Over Time Monthly (last 6 months 2011)

8. Power BI Report & Dashboard

Power BI Desktop was connected to the SSAS cube using a Live Connection. This means the data is not imported into Power BI but is queried directly from the cube in real time. Any changes to the cube are reflected in Power BI after clicking Refresh.

8.1 Connection

Power BI was connected via: Get Data -> Analysis Services -> Server: localhost -> Connect Live.

8.2 Visuals Created

Business Goal	Visual Type	Fields Used
Top-selling products	Bar Chart	Stock Code (Axis), Quantity (Values), Top 10 filter
Sales over time	Line Chart	Year (Axis), Total Amount (Values)
Revenue by country	Map	Country Name (Location), Total Amount (Size)
Total revenue & transactions	Cards	Total Amount, Fact Sales Count
Revenue by region	Pie/Donut Chart	Country Group (Legend), Total Amount (Values)

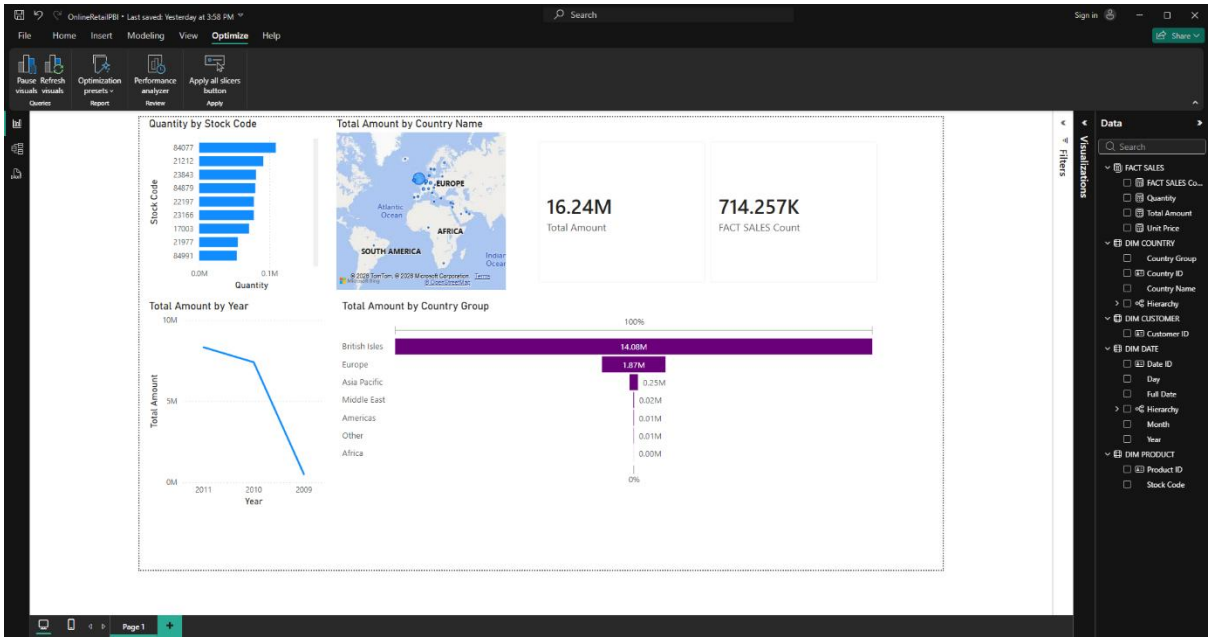

Figure 21- Power BI Full Panel View

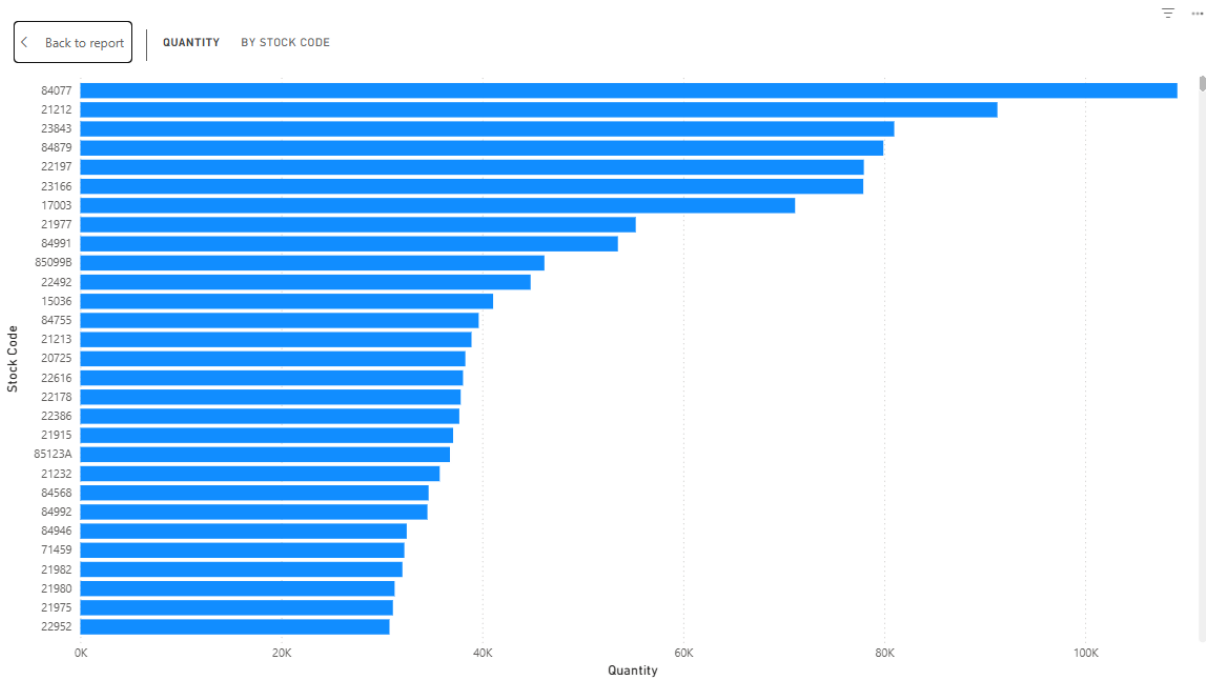

Figure 22- Top products by Quantity

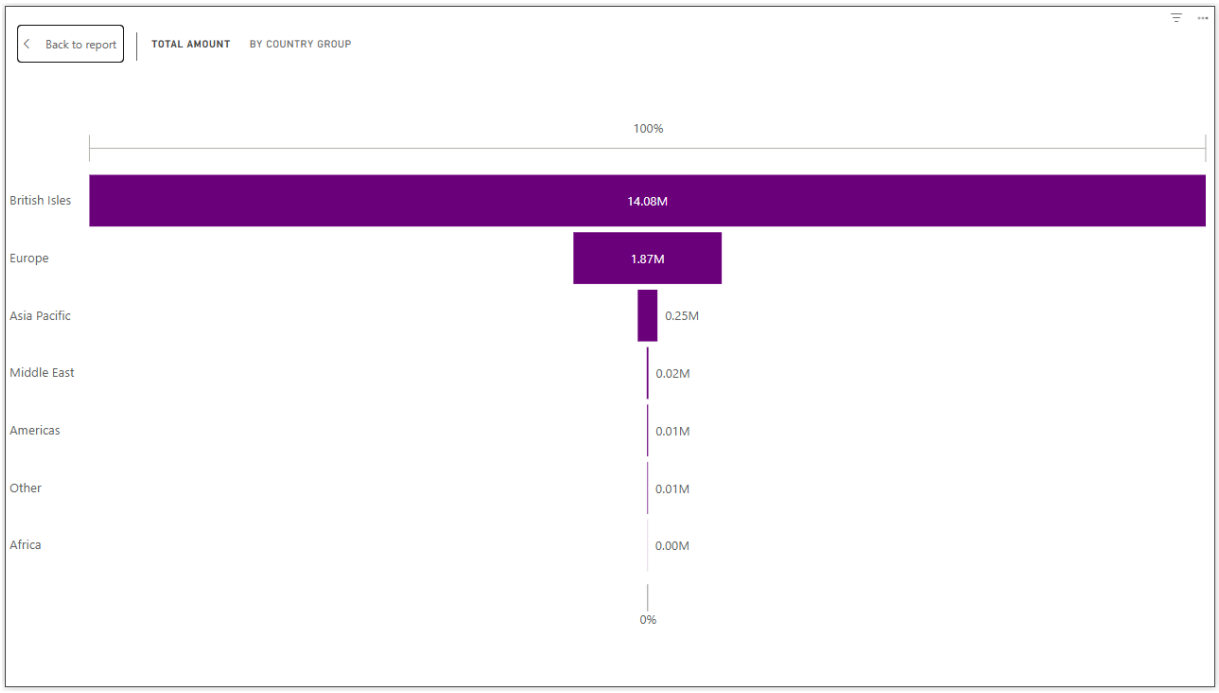

Figure 23- Power BI Pie Chart — Total Amount by Country Group

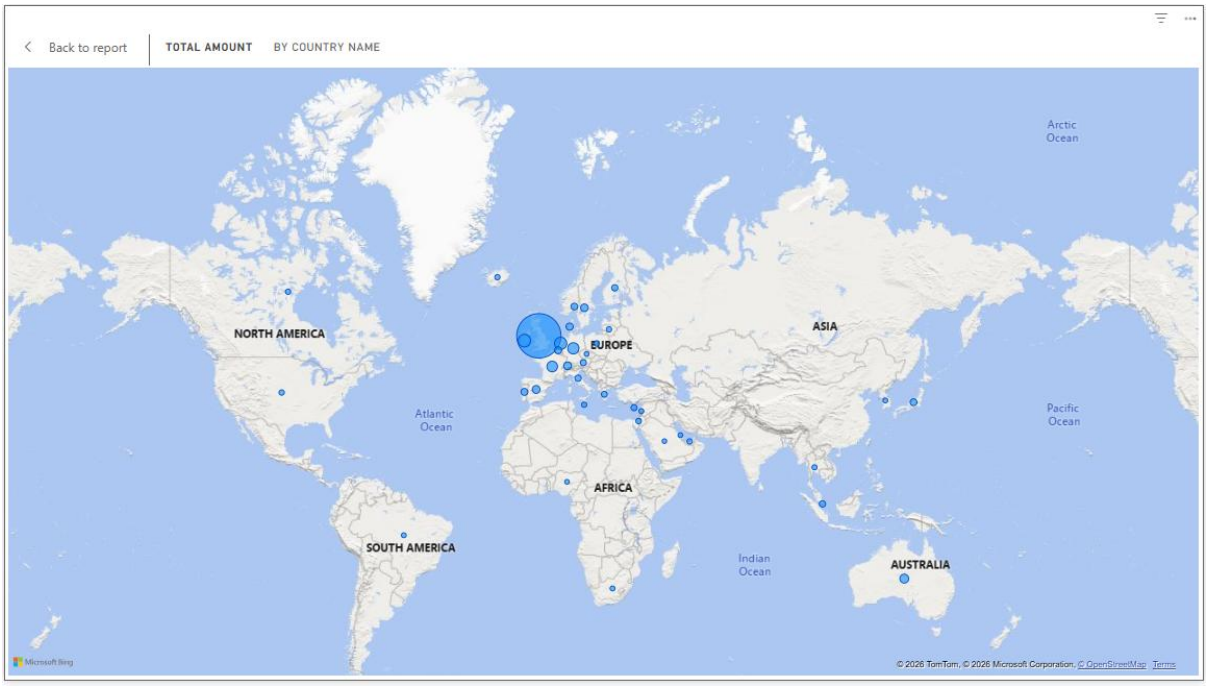

Figure 24- Power BI Map — Revenue by Country

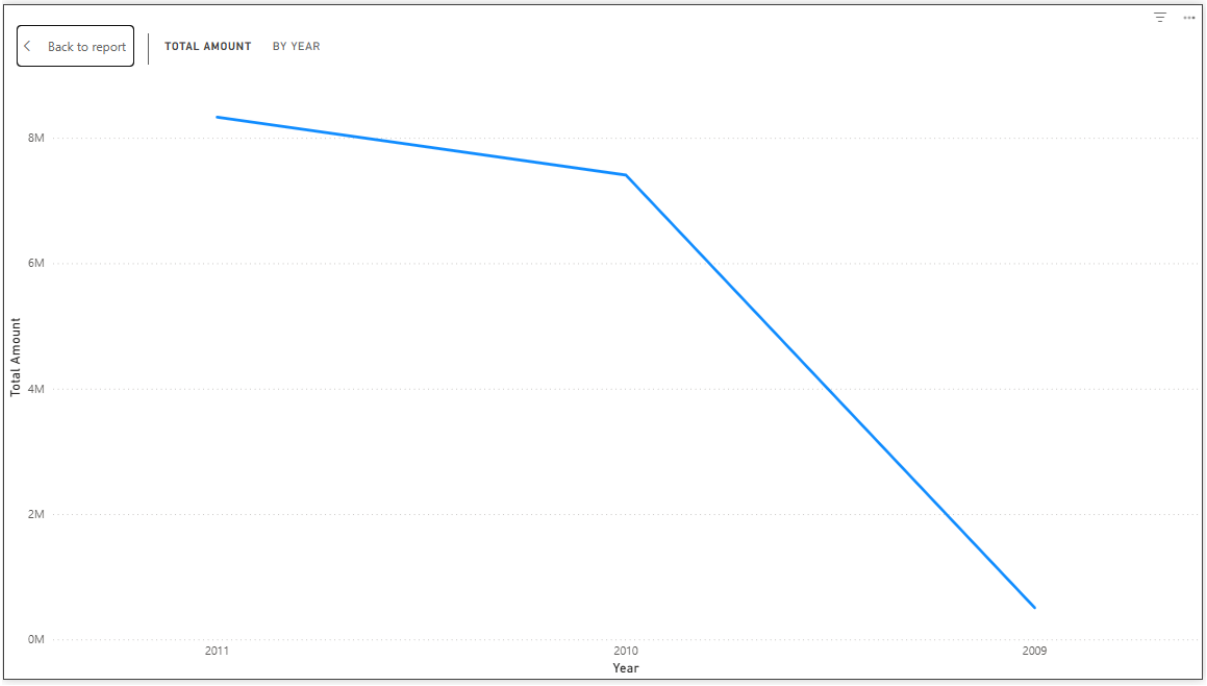

Figure 25-Power BI Line Chart — Total Amount by Year

9. Summary

Throughout this project, we designed and implemented a complete data warehouse pipeline for the Online Retail II dataset, starting from raw transactional Excel data and ending with an interactive Power BI dashboard. We began by designing a star schema in SQL Server, defining dimension and fact tables that reflect the business goals of analyzing sales performance. Using SSIS, we built an ETL pipeline that extracted data from two Excel sheets, merged them using a Union All transformation, cleaned and transformed the data by filtering nulls, negative quantities, cancelled invoices, and duplicates, and loaded incrementally into the warehouse. We then built an OLAP cube in SSAS with dimensions, hierarchies, and attribute relationships, and wrote MDX queries to answer each business question. Finally, we connected Power BI to the cube via a live connection and built a dashboard with five visuals covering product performance, time-based trends, geographic revenue distribution, and regional analysis. Along the way, we learned how to handle real-world data quality issues such as mixed data types in Excel, surrogate key generation, SCD decisions, and the challenges of configuring SSAS in multidimensional mode. The project gave us hands-on experience with the full BI stack and a deeper understanding of how data moves from a raw source all the way to a business-ready report.

The following table summarizes the complete deliverable checklist:

Requirement	Details
ETL Process Description	Sections 3 & 4
Source tables used	STG_OnlineRetail from XLSX
Transformations documented	Derived Column, Lookup, Union All
Null handling	WHERE filters + TRY_CAST
Duplicate handling	GROUP BY + MAX()
Surrogate key generation	IDENTITY on COUNTRY & PRODUCT
SCD handling	Type 1 for DIM_CUSTOMER
All SQL queries documented	Sections 3 & 5
Star Schema filled with data	Section 5.3
OLAP Cube built	Section 6
MDX Queries (5 goals)	Section 7
Power BI Report & Dashboard	Section 8
Data Verification (2+ columns)	Section 5.2 — 4 FK checks